



Building an **AGENTIC** RAG System

What happens when RAG needs to do more than retrieve documents?

Tools. Structured outputs. Retrieval. Self-evaluation.

[Here's how I built the pipeline →](#)

The Tech Stack

Before writing core logic, you need a robust engine.



Google GenAI

LLM + tool-calling reasoning engine.



ChromaDB

Vector database for semantic retrieval.



Pydantic

Validates strict, structured model outputs.



Tiktoken

Token-aware document chunking.

Smart Document Chunking

Large documents don't fit cleanly into an LLM context window.

We use a sliding window approach for token-aware chunking.

Why overlap?

To prevent important context from being lost at chunk boundaries.

✓ Better chunks → Better retrieval

📄 RAW DOCUMENT



[Tokens 001 - 300]

[Tokens 251 - 550]

[Tokens 501 - 800]

Turning Chunks Into Knowledge

01 Embed

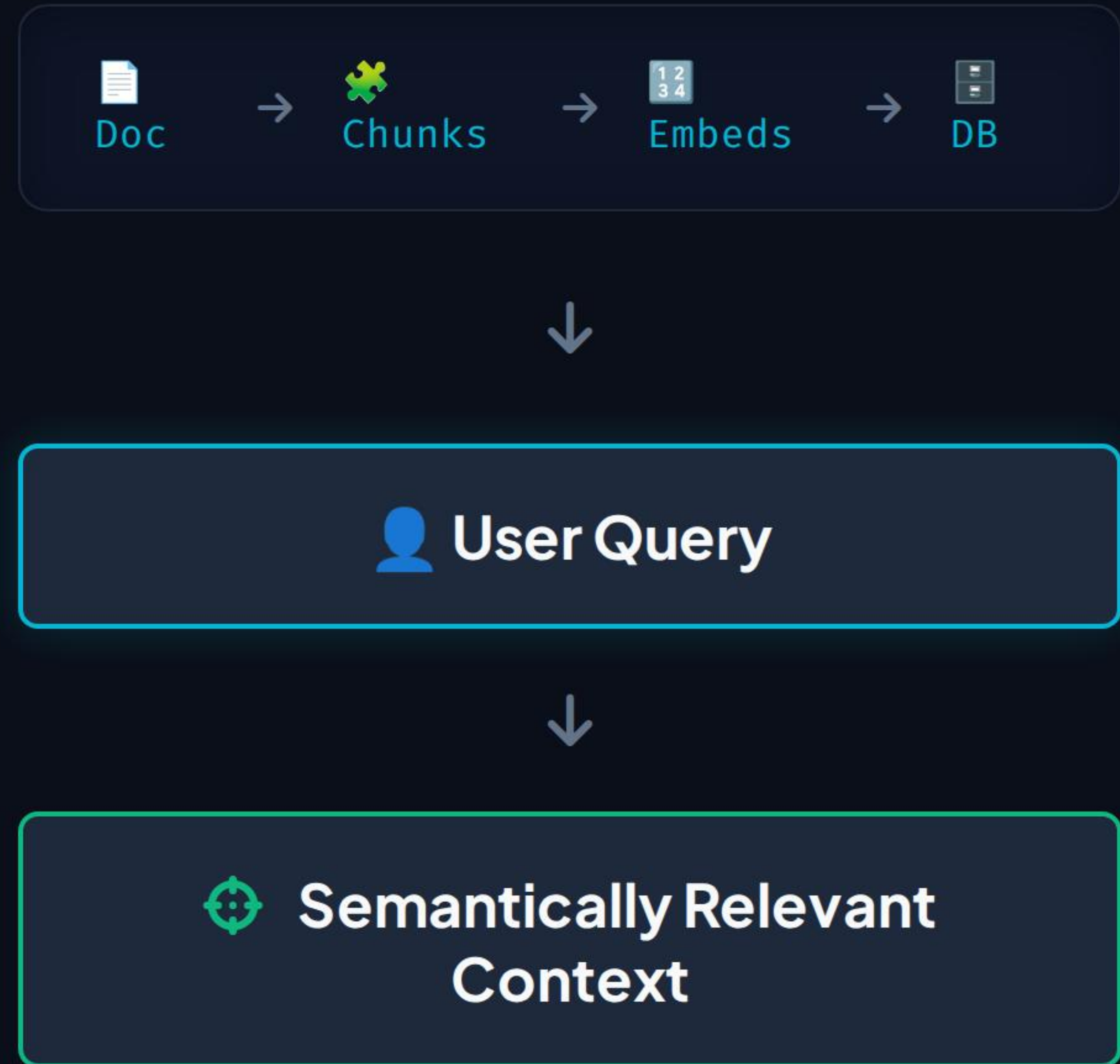
Each text chunk is converted into a mathematical vector representation.

02 Store

ChromaDB stores these vectors alongside their source metadata.

03 Retrieve

User queries retrieve semantically relevant context, not just keyword matches.



Giving the LLM a Calculator

LLMs aren't designed to be perfect calculators.

Instead of forcing the LLM to perform exact arithmetic itself, we give it access to a restricted Python tool for arithmetic expressions.

💡 The LLM independently decides *when* a tool is needed.

👤 USER

"How many years between 1969 and 2026?"



🧠 LLM REASONING

Detects calculation requirement. Routes to tool.



📊 CALCULATOR TOOL

2026 - 1969

57

Taming the LLM with Pydantic

Raw LLM text isn't reliable enough for downstream application logic.

 **Define:** A schema describing the exact expected response.

 **Generate:** The LLM produces structured data.

 **Validate:** Pydantic checks that the response perfectly matches the schema.

 **Predictable outputs → Reliable downstream logic.**

```
{ "answer": "The  
pipeline...", "confidence":  
0.92, "citations": [ {  
  "source": "doc_1.txt",  
  "quote": "RAG allows..." } ]  
}
```

 **Pydantic Validation Passed**

The Core Orchestrator

RAG + Tools + Logic



USER QUERY



 **RETRIEVER**
ChromaDB Context



 **GEMINI LLM**
Reasoning Engine



--

 **Tool Call**

 **Final Answer**

Automated Evaluation Pipeline

How do you know if your RAG system is hallucinating?

Build an automated auditor.

Evaluator LLM (Temp = 0)

- ✓ Faithfulness
- ✓ Citation accuracy
- 🚩 Unsupported claims (Flagged)
- ✓ Reasoning quality

The Final Build

- ✓ Semantic document retrieval
- ✓ Agentic tool calling
- ✓ Structured LLM outputs
- ✓ Citation-aware responses
- ✓ Automated RAG evaluation

The goal wasn't just to make an LLM answer questions.

It was to make the system more reliable, controllable, and testable.

*An automated evaluator flags potential hallucinations and citation problems to ensure systemic reliability.